# Active Learning of Temporal Specifications through Distinguishing Queries

Anonymous Authors

***Abstract*—Temporal specifications are essential for verification and monitoring, yet writing formulas that capture the intended behaviour is difficult. Active learning derives such specifications through interaction between a learner and a teacher, but remains underexplored for temporal logics. Existing approaches require the teacher to answer equivalence queries and to construct counterexamples, and they often rely on SAT-based learning procedures that restart the search after each new example. We present an active-learning framework for LTLf in which the teacher only labels traces and answers natural-language questions about the intended behaviour. The learner maintains an enumerated version space of candidate formulas, generates traces that distinguish candidates, and uses the teacher's hints to rank candidates or to restrict the search. We prove consistency with the observed examples, identification within bounded hypothesis classes with a bound on the number of queries, and a PAC guarantee for the final hypothesis. In experiments with a simulated teacher on $36$ targets, including targets in four standard environments, the learner identifies every target when it compares all rivals, with fewer than seven queries beyond an initial sample of ten traces on average. Correct hints reduce the number of queries by up to $16\%$ and the number of generated candidates by a fifth, and hints used for ranking never prevent identification, even when incorrect. Our enumeration-based version space is faster than the SAT-based approach of previous learners at every target size.**

## I. Introduction

Temporal logics are ubiquitous in formal methods, providing a compact and interpretable language for expressing system requirements in domains including verification [1], [2], runtime monitoring [3], and planning [4]. Recently, they have been employed in the verification of machine learning systems [5], [6], agentic frameworks [7], [8], and robotic systems [9].

However, formalising the intended behaviour in a temporal logic can be a complex and error-prone process, in particular for non-experts [10], [11]. Active learning provides a framework for identifying correct specifications through interaction with a teacher who is assumed to know the target specification. In a typical exact-learning loop, the learner poses membership queries, asking whether a specific execution satisfies the target specification, and equivalence queries, asking whether the current hypothesis is equivalent to the target. For a failed equivalence query, the teacher provides a counterexample on which the hypothesis and target disagree. The learner uses this feedback to refine its hypothesis [12], [13], [14].

Equivalence queries and counterexample construction demand formal expertise that non-expert users may lack. Such users may nevertheless judge whether an execution is acceptable and explain their concerns in natural language, motivating learning from execution labels and informal guidance.

Existing active learners for temporal logics share a second limitation. They encode the search for a consistent formula as a SAT or optimisation problem [15], [16], following SAT-based passive learning [17]. Such a learner either solves the problem anew after each example [15], or computes a small set of candidates once and does not refine them as new examples arrive [16]. Since a formula has many encodings, every solution must be blocked encoding by encoding, and the search becomes slow as the class of formulas grows.

We present a modular active-learning framework for LTLf in which the teacher only labels traces and, when able, answers natural-language questions. The learner reuses its search by maintaining a *version space* of enumerated formulas consistent with the observed labels.

To refine its hypothesis, the learner selects a rival from the version space and asks the teacher to label a trace on which the two disagree. Each label eliminates all inconsistent candidates, allowing efficient learning even from millions of formulas. If no distinguishing trace exists, the rival is equivalent to the hypothesis and is discarded without a query. Our instantiation constructs the version space using grammar-based enumeration [18] and generates distinguishing traces using automata [19] or formula progression [4]. SAT-based approaches can also be incorporated. Traces may be *free*, allowing arbitrary event sequences, or constrained by an *environment*, ensuring that the teacher only labels feasible executions.

Membership labels indicate whether an execution is acceptable, but not why. To provide this additional information, our framework introduces *natural-language queries* at two levels. At Level 1, the teacher identifies the type of mismatch, such as a missing event or incorrect ordering. At Level 2, the teacher specifies the requirement more precisely, identifying the events involved and their intended relationship. Both levels guide the learner in restricting the candidate space, potentially reducing the number of queries needed.

**Example 1** (Motivating example)**.** *To illustrate the framework, consider a mobile service robot that picks up objects and delivers them. The user knows the intended behaviour but cannot write it in temporal logic. Over* $\mathsf{AP} = \{pickup, deliver\}$*, the target is* $\varphi_T = \mathbf{G}\big(pickup \rightarrow \mathbf{X}(\neg pickup\, \mathbf{U}\, deliver)\big)$ *: every pickup is followed later by a delivery, with no other pickup in between. We follow a run of our learner with a simulated teacher, started from two labelled traces.*

*First, the user labels two executions:* $\{pickup\};\{deliver\}$ *is accepted and* $\{pickup, deliver\}$ *is rejected. Asked whether*

*something required is missing, the user answers that the object should be delivered after a pickup; this hint becomes the requirement* $h = \mathbf{G}(pickup \rightarrow \mathbf{F}\, deliver)$*, and the learner first searches the formulas consistent with it. Then, the learner repeatedly compares its hypothesis with other consistent formulas and asks the user about an execution on which they disagree. For instance, when the current hypothesis is* $\mathbf{G}(pickup \rightarrow \mathbf{FX}\, deliver)$ *and the rival is* $\varphi_T$*, the user rejects the distinguishing trace* $\{pickup\};\{pickup\};\{deliver\}$*. This refutes the hypothesis, so the rival* $\varphi_T$ *becomes the new hypothesis. Finally, when no remaining formula can be told apart from the hypothesis, the learner returns* $\varphi_T$*, after* $16$ *membership queries including the two initial ones; without the hint, it needs* $20$*.*

Our contributions therefore are:

- a modular active-learning framework for LTLf that uses membership queries and natural-language hints (Section IV);
- different levels of natural-language hints to focus on relevant formulas only (Section IV-D);
- guarantees of sample consistency, identification within bounded hypothesis classes with a bound on the number of queries, and a PAC guarantee for the final hypothesis (Section V);
- an open-source modular Python implementation with replaceable procedures for version-space construction and distinguishing-trace generation (Section VI), evaluated over $5{,}540$ runs on $36$ LTLf targets expressing specification patterns used in practice [20], including $16$ targets across four standard environments (Section VII);
- experimental evidence that the learner identifies all targets when comparing all rivals, requiring on average only $6.2$ additional queries over free traces and $2.5$ in environments, even with up to $5.1$ million candidate formulas. Correct hints reduce queries by up to $16\%$, while our version-space construction outperforms the SAT-based alternative ($8.8\,$s versus $22.4\,$s for size-7 targets).

Technical details, including proofs and the experimental details, are in the appendices of the full version of this paper, available in our repository at https://anonymous.4open.science/r/ltl-active-learning-with-hints-585D/.

### A. Related work

Active learning has its roots in automata learning, notably Angluin's $L^*$ algorithm [12], which has been widely applied to learning behavioural models of software and hardware systems [14]. Automata learning benefits from the Myhill–Nerode relation, which connects observed behaviour to the structure of the learned model. This connection is less direct for logical formalisms, including temporal logics, motivating separate approaches to actively learning logical specifications.

Three works on active learning of temporal specifications are particularly close to ours. Camacho and McIlraith [15] learn minimal LTLf formulas using equivalence queries. Their approach repeatedly invokes a SAT-based passive learner to find the smallest formula consistent with the accumulated examples, incorporating counterexamples returned by the oracle after each failed equivalence query. Linard and Tůmová [21] learn STL specifications using membership and equivalence queries. Their learner maintains an online decision tree, generates signals for membership queries, and evaluates equivalence through a distance to the target, obtaining labelled counterexamples when equivalence fails. Gavran et al. [16] synthesise LTL specifications for robotic tasks from an example trace and a natural-language description. They use an optimisation-modulo-theories encoding to generate candidate specifications and ask users to assess behaviours that distinguish them.

Our framework differs from these works in three main respects. First, the teacher only labels traces and answers natural-language questions. Unlike in [15], [21], the teacher need not evaluate hypotheses or construct counterexamples. Second, the learner maintains all consistent formulas within a bounded class, guaranteeing identification of the target within a bounded number of queries (Section V). In contrast, the restricted candidate set of [16] may exclude the target, while convergence in [21] is demonstrated experimentally but not formally established. Third, our version space is constructed once and incrementally filtered as new labels arrive, whereas [15] solves a new SAT problem after each counterexample. Our experiments show that enumeration is faster than SAT-based construction across all evaluated target sizes.

Natural language also serves a different purpose. While [16] uses an initial natural-language description of the task, our framework elicits partial requirements through optional natural-language queries during learning, complementing membership labels. Finally, the framework is modular, allowing individual procedures to be replaced. For instance, a SAT-based version space can reduce memory consumption for larger hypothesis classes (Section VII-C).

A closely related area is *passive learning*, which infers formulas from fixed datasets and often serves as a subroutine in active learning. For LTL and LTLf, notable approaches include SAT-based inference [17], scalable anytime learning [22], graph-neural-network-based inference [23], learning from positive examples [24], statistical inference from event logs [25], and grammar-based enumeration with search heuristics [18], which forms the basis of our version-space construction. For STL, existing methods include decision-tree learning [26], neural-symbolic approaches [27], and optimisation-based decision-tree learning [28]. More recently, Baharisangari et al. [29] have applied STL inference to causal discovery from time-series data.

## II. Preliminaries

### A. Linear Temporal Logic on Finite Traces

To illustrate the framework, we use linear temporal logic on finite traces (LTLf) and introduce the necessary definitions [19]. We represent relevant system properties by a finite set of atomic propositions $\mathsf{AP}$. The alphabet $\Sigma = 2^{\mathsf{AP}}$ consists of all subsets of $\mathsf{AP}$; each letter specifies which propositions hold at a given

position. The trace domain $\mathcal{W}$ is $\Sigma^*$, which includes the empty trace $\varepsilon$, unless an environment restricts it to feasible traces (Section V).

The syntax of LTLf formulas is defined by the grammar

$$\varphi ::= p \in \mathsf{AP} \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathbf{X}\varphi \mid \varphi\,\mathbf{U}\,\varphi.$$

We also use the standard derived Boolean operators $\vee$ and $\rightarrow$, constants $\mathrm{false}$ and $\mathrm{true}$, and the temporal operators $\mathbf{F}$ (eventually) and $\mathbf{G}$ (always). Although our framework supports formulas of arbitrary syntactic structure, we restrict negation to atomic propositions, following the equivalent negation normal form used in [18], to simplify the search.

For a finite trace $w$ and an index $0 \le i \le |w|$, satisfaction is defined as follows:

$$\begin{aligned}
w, i \models p &\quad \text{iff } i < |w| \text{ and } p \in w_i,\\
w, i \models \neg\varphi &\quad \text{iff } w, i \not\models \varphi,\\
w, i \models \varphi \wedge \psi &\quad \text{iff } w, i \models \varphi \text{ and } w, i \models \psi,\\
w, i \models \mathbf{X}\varphi &\quad \text{iff } i + 1 < |w| \text{ and}\\
&\qquad\qquad w, i + 1 \models \varphi,\\
w, i \models \varphi\,\mathbf{U}\,\psi &\quad \text{iff there is } j \ge i \text{ with } j < |w|,\ w, j \models \psi,\\
&\qquad \text{and } w, k \models \varphi \text{ for every } i \le k < j.
\end{aligned}$$

We write $w \models \varphi$ for $w, 0 \models \varphi$ and $L(\varphi) = \{w \in \mathcal{W} \mid w \models \varphi\}$. We define equivalence and implication through the corresponding languages: $\varphi \equiv \psi$ means $L(\varphi) = L(\psi)$, and $\varphi \models \psi$ means $L(\varphi) \subseteq L(\psi)$.

Following [18], we measure formula complexity by size and temporal depth. The size $|\varphi|$ is the number of syntax-tree nodes, counting each literal (including negated propositions) as a single leaf. The temporal depth $\mathrm{td}(\varphi)$ is the maximum number of temporal operators along any root-to-leaf path. For example, $\mathbf{G}(p \rightarrow \mathbf{F}(\neg q))$ has size 5 and temporal depth 2.

### B. Background on Active Learning

Active learning [12] involves two entities: a learner and a teacher. The teacher represents a non-expert user who knows the intended target specification $\varphi_T$ and responds to the learner's queries. The learner interacts with the teacher through two types of queries, formalised below: membership queries, which request the teacher to classify traces according to the target specification, and natural-language queries, which elicit hints to guide the refinement of the learned specification. Note that, in contrast to traditional active learning, our framework avoids equivalence queries, as non-expert users cannot be expected to assess whether a learned specification precisely captures their intentions.

**Definition 1** (Membership query)**.** *Given a target $\varphi_T$ and a trace $w \in \mathcal{W}$, a membership query $\mathsf{MQ}(w)$ returns 1 if $w \models \varphi_T$ and 0 otherwise.*

**Definition 2** (Natural-language query)**.** *Given a target $\varphi_T$, a natural-language query returns a hint $h$ describing a property of $\varphi_T$.*

The traces labelled so far and their labels form a labelled *sample* $S \subseteq \mathcal{W} \times \{0, 1\}$. A formula $\varphi$ is *consistent* with $S$ if, for every $(w, y) \in S$, $w \models \varphi$ if and only if $y = 1$.

## III. Learning Problem

Fix a finite set of propositions $\mathsf{AP}$ and the trace domain $\mathcal{W} \subseteq (2^{\mathsf{AP}})^*$. Let $\mathcal{L}_{n,d}$ denote the finite class of LTLf formulas over $\mathsf{AP}$ with size at most $n$ and temporal depth at most $d$ (Section IV-A). The teacher answers membership queries according to an unknown target $\varphi_T \in \mathcal{L}_{N,D}$, where $N, D$ are unknown to the learner.

The objective is to identify a hypothesis $\varphi_H$ using as few membership queries as possible, optionally guided by natural-language hints. Every hypothesis must be consistent with the labelled sample $S = \{(w, \mathsf{MQ}(w)) \mid w \text{ was queried}\}$. We distinguish two objectives: (i) *exact identification*, $\varphi_H \equiv \varphi_T$, when the searched class contains $\varphi_T$ and refinement is exhaustive; and (ii) *approximate identification* under a distribution $\mathcal{D}$ over $\mathcal{W}$, measured by

$$\mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) = \Pr_{w \sim \mathcal{D}}[\mathbf{1}[w \models \varphi_H] \neq \mathbf{1}[w \models \varphi_T]].$$

For $\epsilon, \delta \in (0, 1)$, certification must satisfy

$$\Pr[\text{accept } \varphi_H \ \wedge\ \mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) > \epsilon] \le \delta.$$

This guarantee does not imply exact equivalence.

## IV. Active Learning Framework

The learner starts from an initial hypothesis and refines it iteratively, using several interchangeable procedures (Algorithm 1).

Initialize($L$) labels $L$ initial traces (line 1), sampled uniformly, generated as feasible environment rollouts, or provided as input. The learner then searches a *class schedule*, an increasing sequence of finite classes $\mathcal{L}_1 \subseteq \cdots \subseteq \mathcal{L}_M$ that fixes the order in which formulas are considered; by default, $\mathcal{L}_m = \mathcal{L}_{m,d}$ for a fixed depth $d$, so smaller formulas come first. For each class, VersionSpace adds formulas consistent with the labelled sample $S$ to the version space $V$ (line 4; Section IV-A), and the smallest formula in $V$ becomes the initial hypothesis (line 6). AskHint may request natural-language hints to reorder rivals or adjust the class schedule (line 7; Section IV-D).

In each refinement round (lines 8–21), NextRival orders the remaining candidates, called *rivals*, and the learner considers the first $K$ (Section IV-B). If Distinguish finds no distinguishing trace, the rival is equivalent to $\varphi_H$ and is discarded without a query (line 13; Section IV-C). Otherwise, the teacher labels the returned trace $w$ (line 15). If the label refutes $\varphi_H$, the rival becomes the new hypothesis (line 16), and all inconsistent formulas are removed from $V$ (line 17). A round without a query ends refinement for the current class. Finally, Certify evaluates the hypothesis on $P$ fresh traces (line 24; Section IV-E). The implementation also limits the number of comparisons (Section VI).

**Algorithm 1** Active learning by distinguishing queries

**Require:** Class schedule $\mathcal{L}_1 \subseteq \cdots \subseteq \mathcal{L}_M$, shortlist size $K \in \mathbb{N} \cup \{\infty\}$, budgets $L, P$
**Ensure:** Hypothesis $\varphi_H$, labelled sample $S$, certification result $b$ and counterexample $w$

1: $S \leftarrow \textsc{Initialize}(L)$
2: $\mathcal{H}_{\text{hint}} \leftarrow \emptyset$; $\varphi_H \leftarrow \bot$; $V \leftarrow \emptyset$; $\mathcal{L}_0 \leftarrow \emptyset$ ▷ $\bot$: no hypothesis yet
3: **for** $m = 1, \ldots, M$ **do**
4: $V \leftarrow V \cup \textsc{VersionSpace}(\mathcal{L}_m \setminus \mathcal{L}_{m-1}, S)$
5: **if** $V = \emptyset$ **then continue**
6: **if** $\varphi_H = \bot$ **then** $\varphi_H \leftarrow$ a smallest formula in $V$
7: $\mathcal{H}_{\text{hint}} \leftarrow \mathcal{H}_{\text{hint}} \cup \textsc{AskHint}(\varphi_H)$ ▷ at most once
8: **repeat**
9: $R \leftarrow$ first $K$ formulas of $\textsc{NextRival}(V, \varphi_H, \mathcal{H}_{\text{hint}})$
10: **for all** $\psi \in R$ **do**
11: $w \leftarrow \textsc{Distinguish}(\varphi_H, \psi)$
12: **if** $w =$ false **then**
13: $V \leftarrow V \setminus \{\psi\}$ ▷ $\psi \equiv \varphi_H$
14: **else**
15: $S \leftarrow S \cup \{(w, \mathsf{MQ}(w))\}$
16: **if** $\varphi_H$ is inconsistent with $S$ **then** $\varphi_H \leftarrow \psi$
17: $V \leftarrow \{\varphi \in V \mid \varphi \text{ is consistent with } S\}$
18: **break** ▷ Start the next round
19: **end if**
20: **end for**
21: **until** no query was asked in this round
22: **end for**
23: **if** $\varphi_H = \bot$ **then return** $(\textsc{Failure}, S)$
24: $(b, w) \leftarrow \textsc{Certify}(\varphi_H, P)$
25: **return** $(\varphi_H, S, b, w)$

*Correctness and cost.:* Each query refutes either $\varphi_H$ or the compared rival, eliminating at least one candidate without removing the target. Thus, the hypothesis remains consistent with all observed labels (Proposition 1). When all rivals are compared ($K = \infty$, the *exhaustive rule*), the learner identifies a hypothesis equivalent to the target (Theorem 1), or equivalent relative to an environment (Theorem 2). Comparing only the first $K$ rivals (the *top-$K$ rule*) does not guarantee equivalence, but the certification bound (Theorem 3) still applies. Under the exhaustive rule, the choice of version space and rival ordering affects only query efficiency and running time.

We now describe each procedure in turn.

### A. Version space

For a class $\mathcal{L}$ and a sample $S$, the version space is

$$V_{\mathcal{L}}(S) = \{\psi \in \mathcal{L} \mid w \models \psi \text{ iff } y = 1 \text{ for all } (w, y) \in S\}.$$

$\textsc{VersionSpace}(\mathcal{L}, S)$ returns $V_{\mathcal{L}}(S)$. Algorithm 1 calls it only for the formulas that are new in the schedule (line 4), and it keeps the version space up to date after each label (line 17). We describe two implementations.

*a) Enumeration.:* Following the grammar-based enumeration of Roy et al. [18], in the first approach, we build the version space bottom-up by dynamic programming. Starting from the atomic propositions and their negations, the formulas of each size and nesting depth are built from smaller ones with the operators $\mathbf{X}$, $\mathbf{F}$, $\mathbf{G}$, $\mathbf{U}$, $\wedge$, $\vee$ and $\rightarrow$. A new formula enters the version space only if it is consistent with the current $S$.

As in [18], two heuristics discard redundant formulas. *Temporal simplification* discards the forms $\mathbf{FF}\varphi$, $\mathbf{GG}\varphi$, $\mathbf{X}\varphi\,\mathbf{U}\,\mathbf{X}\psi$, $\mathbf{G}\varphi \wedge \mathbf{G}\psi$, $\mathbf{X}\varphi \wedge \mathbf{X}\psi$, $\mathbf{F}\varphi \vee \mathbf{F}\psi$ and $\mathbf{X}\varphi \vee \mathbf{X}\psi$, each equivalent over finite traces to a smaller formula. *Boolean simplification* discards binary formulas whose operands are syntactically equal or complementary literals. Since larger formulas are built from smaller ones, all formulas of size below $n$ are stored, consistent or not, so time and memory grow with $|\mathcal{L}_{n,d}|$.

*b) SAT-based construction.:* Our second approach follows SAT-based passive learning [17], [15] and represents the version space implicitly by a propositional formula $\Phi^S_{m,d} = \phi_{\text{syn}} \wedge \phi_{\text{sem}}$, whose satisfying assignments encode the formulas of size $m$ and temporal depth at most $d$ that are consistent with $S$: $\phi_{\text{syn}}$ encodes their syntax and $\phi_{\text{sem}}$ their semantics on the sample traces.

For $\phi_{\text{syn}}$, a formula of size $m$ is represented by its syntax tree with nodes $1, \ldots, m$: the root is node $m$, children have smaller indices than their parents, leaves are propositions or their negations, and inner nodes are operators. The variable $x_{i,\lambda}$ indicates that node $i$ is labelled by the literal or operator $\lambda$, and $l_{i,j}$ and $r_{i,j}$ indicate that node $j$ is the left or right child of node $i$. The constraints assign exactly one label to each node and give each operator as many children as its arity. Unlike the syntax DAGs of [17], every non-root node has exactly one parent, so the encoding represents syntax trees and matches our size measure. In addition, the variable $e_{i,k}$ indicates that the subformula rooted at node $i$ has temporal depth at most $k$, and the constraints propagate these values from the leaves to the root, so that the root has temporal depth at most $d$.

For $\phi_{\text{sem}}$, each trace $w$ of $S$ is evaluated position by position. The variable $z^w_{i,k}$ indicates that the subformula rooted at node $i$ holds at position $k$ of $w$, for $0 \le k \le |w|$, where $k = |w|$ is the end position. The constraints relate these variables to the labels and children of the nodes, following the finite-trace semantics of the preliminaries, and require $z^w_{m,0} = y$ for each $(w, y) \in S$. Thus $\Phi^S_{m,d}$ is satisfiable if and only if some formula of size $m$ and temporal depth at most $d$ is consistent with $S$, and such a formula can be decoded from any satisfying assignment.

In this approach, the version space need not be listed explicitly. $\textsc{NextRival}$ asks an incremental SAT solver for models of $\Phi^S_{m,d}$, in increasing order of $m$. If $\textsc{Distinguish}$ proves the decoded rival $\psi$ equivalent to $\varphi_H$, a blocking clause, as in [17], [16], excludes the current assignment to the structural variables $x_{i,\lambda}$, $l_{i,j}$ and $r_{i,j}$.

### B. Next rival

$\textsc{NextRival}(V, \varphi_H, \mathcal{H}_{\text{hint}})$ lists the formulas of $V \setminus \{\varphi_H\}$ in some order, and Algorithm 1 compares $\varphi_H$ with the first

$K$ of them. The order decides which formulas are compared, and hence which traces are queried. We consider three orders.

*a) Enumeration.:* The rivals come in the order in which the version space produces them: smallest formulas first, with ties broken by a fixed textual order. With the SAT-based construction, this is the order in which the solver returns models, again smallest size first. The order needs no extra computation.

*b) Semantic.:* The rivals closest to $\varphi_H$ in behaviour come first. For a probe set $U = \{u_1, \ldots, u_r\}$, the semantic distance

$$d_{\mathrm{sem}}(\varphi, \psi; U) = \frac{1}{r} \sum_{i=1}^{r} \mathbf{1}[\mathbf{1}[u_i \models \varphi] \neq \mathbf{1}[u_i \models \psi]]$$

is the fraction of probe traces on which the two formulas disagree. The probe set is drawn once per run, independently of the sample, from the distribution of the initial traces, or as feasible rollouts in an environment, so $d_{\mathrm{sem}}$ estimates the probability that the two formulas disagree on such a trace. We also evaluate the reverse order, *semantic-far*, which puts the farthest rivals first.

*c) Split.:* We prioritise rivals whose witnesses split the version space as evenly as possible. A more balanced split guarantees that more candidates are eliminated, regardless of how the teacher labels the trace.

For each rival $\psi$, let $w_\psi$ be the first trace in breadth-first order in the short-trace pool $T_\ell$ (Section IV-C) on which $\psi$ and $\varphi_H$ disagree, if such a trace exists. Let $n_\psi = |\{\varphi \in V \mid w_\psi \models \varphi\}|$ count the candidates that accept $w_\psi$. We rank rivals by increasing imbalance $\big|2n_\psi - |V|\big|$. Rivals with no witness in $T_\ell$ come last, in enumeration order.

### C. Distinguish

DISTINGUISH$(\varphi, \psi)$ returns a trace in $\mathcal{W}$ on which $\varphi$ and $\psi$ disagree, or $\mathrm{false}$ if no such trace exists. One witness suffices per comparison because either label makes one of the two formulas inconsistent with the sample.

Before running either of the two sound and complete procedures below, we scan a fixed pool $T_\ell$ containing all traces of length at most $\ell$, restricted to feasible traces when an environment is given. We scan the pool in breadth-first order and return the first trace on which the formulas disagree. If the pool contains no witness, we run the chosen procedure.

*1) Distinguishing with automata:* The first procedure translates each LTLf formula $\varphi$ into a deterministic finite automaton (DFA) $\mathcal{A}_\varphi$ with the same language [19]. For candidates $\varphi$ and $\psi$, the product construction [30], in which a state is accepting exactly when one of its components is, yields a DFA $\mathcal{A}_\Delta$ with $L(\mathcal{A}_\Delta) = L(\mathcal{A}_\varphi) \triangle L(\mathcal{A}_\psi)$. Every trace of $L(\mathcal{A}_\Delta)$ is accepted by exactly one candidate, so the procedure returns such a trace if $\mathcal{A}_\Delta$ is nonempty, and $\mathrm{false}$ otherwise.

*2) Distinguishing by formula progression:* The automata-based procedure builds an automaton for every compared formula. Our implementation instead explores the symmetric-difference automaton on the fly, using formula progression [4]. For a letter $a$, the progression $\mathrm{prog}(\varphi, a)$ is the formula that the rest of the trace must satisfy once $a$ has been read; it extends to traces letter by letter. The *end value* $\mathrm{fin}(\varphi) \in \{0, 1\}$ states whether $\varphi$ holds on the empty trace. Both are computed syntactically, and $uw \models \varphi$ if and only if $w \models \mathrm{prog}(\varphi, u)$. Appendix A gives the definitions.

**Example 2.** *Let* $\mathsf{AP} = \{a, b\}$ *and* $\varphi = a\,\mathbf{U}\,b$. *Then* $\mathrm{prog}(\varphi, \{a\}) = \varphi$, $\mathrm{prog}(\varphi, \{b\}) = \mathrm{true}$ *and* $\mathrm{prog}(\varphi, \emptyset) = \mathrm{false}$, *while* $\mathrm{fin}(\varphi) = 0$ *and* $\mathrm{fin}(\mathrm{true}) = 1$. *Hence* $\{a\}\{b\} \models \varphi$, *and* $\{a\} \not\models \varphi$. *To compare* $\varphi$ *with* $\psi = \mathbf{F}b$, *for which* $\mathrm{prog}(\psi, \emptyset) = \psi$ *and* $\mathrm{prog}(\psi, \{b\}) = \mathrm{true}$, *the trace* $\emptyset\{b\}$ *progresses the pair* $(\varphi, \psi)$ *to* $(\mathrm{false}, \mathrm{true})$, *whose end values differ, while no shorter trace distinguishes* $\varphi$ *from* $\psi$.

The procedure is a breadth-first search over pairs of progressed formulas. It starts from $(\varphi, \psi)$ with the empty trace. From a pair reached by a trace $u$, each letter $a$ leads to the pair progressed by $a$, reached by $ua$. The search returns $u$ as soon as the two end values differ. It visits each pair once and does not expand a pair with equal components, since no extension of $u$ distinguishes the formulas there. We keep progressed formulas in a *normal form*, as sets of clauses over the subformulas of the input, so only finitely many pairs arise and the search terminates. In the worst case it visits doubly exponentially many pairs in $|\varphi| + |\psi|$; in practice it builds only the pairs it reaches and stops at the first witness. The implementation bounds the number of pairs (Section VI); a search that exceeds the bound reports a failure instead of an answer.

**Lemma 1** (Correctness of DISTINGUISH)**.** *Let* $\varphi$ *and* $\psi$ *be LTLf formulas, and let* DISTINGUISH *be the automata-based or the progression-based procedure, without a budget. Then* DISTINGUISH$(\varphi, \psi)$ *terminates, and it returns a trace if and only if some* $w \in \Sigma^*$ *satisfies* $(w \models \varphi) \oplus (w \models \psi)$*; every returned trace satisfies this condition. Hence it returns* $\mathrm{false}$ *if and only if* $\varphi$ *and* $\psi$ *are equivalent over* $\Sigma^*$. *The progression-based procedure returns a shortest such trace.*

The proof is given in Appendix B.

*a) Environments.:* For an environment given as a finite labelled transition system, the progression-based search runs over triples of an environment state and two progressed formulas, and follows the transitions of the environment instead of all letters. There are finitely many triples, so the search returns a nonempty feasible distinguishing trace if and only if one exists. During learning, the implementation also bounds the number of actions (Section VI); a search that reaches this bound returns no answer, which preserves soundness but not completeness.

### D. Natural-language hints

A hint gives the learner a *partial requirement*: a property that every acceptable execution has, but that need not describe the whole target. Hints never change a label. They act on Algorithm 1 in two ways only: by reordering the rivals in NEXTRIVAL, or by restricting the class schedule of VERSION-SPACE. An execution shown to the teacher may prompt a hint, but the learner uses only the answer. The learner asks at two

TABLE I

QUESTIONS AT BOTH LEVELS AND THE FORMULA FAMILIES $\mathcal{H}_c$ OF THE QUESTION TYPES. LEVEL 1 ANSWERS ARE YES, NO, OR UNSURE; LEVEL 2 IS ASKED AFTER A YES ANSWER. $B$ AND $B'$ STAND FOR BOOLEAN FORMULAS. IN OUR EXPERIMENTS, A SIMULATED TEACHER ANSWERS.

| Type | Level 1 question | Level 2 question | Patterns in $\mathcal{H}_c$ |
|---|---|---|---|
| Undesired behaviour | Does anything happen that should not happen? | Which event should not happen, and when? | $\mathbf{G}B$, $\mathbf{G}\neg B$, $\mathbf{G}\neg(B \wedge B')$ |
| Missing behaviour | Is something required missing? | Which event is missing, and when is it required? | $\mathbf{F}B$, $\mathbf{G}(B \rightarrow \mathbf{F}B')$, $\mathbf{G}(B \rightarrow \mathbf{XF}B')$ |
| Ordering | Do any events happen in the wrong order? | Which events, and in what order? | $B\,\mathbf{U}\,B'$, $\mathbf{F}(B \wedge \mathbf{F}(B' \wedge \cdots))$, $\mathbf{F}(B \wedge \mathbf{XF}(B' \wedge \cdots))$, $\mathbf{G}(B \rightarrow \mathbf{F}B')$, $\mathbf{G}(B \rightarrow \mathbf{XF}B')$, $\mathbf{G}(B \rightarrow \mathbf{X}B')$ |
| Completion | Is any task left unfinished? | Which task, and what would complete it? | $\mathbf{F}B$, $\mathbf{G}(B \rightarrow \mathbf{F}B')$, $\mathbf{G}(B \rightarrow \mathbf{XF}B')$, $\mathbf{FG}B$ |
| Repetition | Does anything happen too often or not often enough? | Which event, and how should its repetition change? | $\mathbf{GF}B$ |

levels (Table I): at Level 1 the teacher selects a question type $c$, and at Level 2 they state a requirement, which is translated into an LTLf formula $h$.

*a) Level 1: question types.:* Each question type $c$ stands for a set of specification patterns [20], and $\mathcal{H}_c$ is the family of formulas that are instances of these patterns (Table I). For example, missing behaviour stands for eventuality and response, with instances such as $\mathbf{F}a$ and $\mathbf{G}(b \rightarrow \mathbf{F}a)$. The families overlap, since one requirement can be described by different question types. The learner uses only the question types answered with Yes.

*b) Level 2: a containing language.:* The Level-2 answer names the events and conditions of a requirement. It is translated into a formula $h$ that every acceptable execution should satisfy, for example by a large language model; we assume the translation is given and do not evaluate it. A correct hint satisfies $\varphi_T \models h$, that is, $L(\varphi_T) \subseteq L(h)$. For example, "every pickup must be followed by a delivery" gives $h = \mathbf{G}(\mathit{pickup} \rightarrow \mathbf{F}\,\mathit{deliver})$. This hint is correct for the target of the motivating example (Example 1), but it does not determine that target.

*c) Using hints.:* ASKHINT (line 7 of Algorithm 1) asks the Level-1 question at most once in a run. The families $\mathcal{H}_c$ of the question types answered with Yes are the *hinted families*. The learner uses them in one of two ways.

*Ranking.* NEXTRIVAL sorts the rivals stably by their distance to the hinted families: the smallest number of syntactic changes that turn the rival into an instance of a hinted family. Ranking changes only the order in which rivals are compared.

*Pruning.* The class schedule is replaced by levels: first the formulas of $\mathcal{L}_{n,d}$ in the hinted families, then those whose root operator is the root operator of some formula in a hinted family, and finally $\mathcal{L}_{n,d}$. Each level is searched by size, as in the default schedule. A Level-2 hint $h$ adds a first level that contains the formulas $\psi$ rejecting every trace of $T_\ell$ that $h$ rejects, which is the check $\psi \models h$ restricted to $T_\ell$. The learner moves to the next level only when the current level leaves the version space empty. Since pruning changes the class schedule, its hints are asked before the first call of VERSIONSPACE (line 4) rather than at line 7.

Hints never change a label, and within the searched class the version space is filtered by labels only, so every hypothesis remains consistent with $S$ whatever the accuracy of the hints (Proposition 1). Ranking excludes no candidate, so an incorrect ranking hint can only lead to less useful queries. Pruning excludes candidates: if an incorrect hint leaves a consistent formula in a level without $\varphi_T$, the learner ends in that level. If every hint is correct, $\varphi_T$ lies in the first level, and Theorem 1 applied to that level shows that the exhaustive rule still ends with $\varphi_H \equiv \varphi_T$.

### E. Certify

We assess distributional error in the probably approximately correct learning framework [31], using a binomial test of a fixed hypothesis [32]. The final hypothesis is frozen. CERTIFY$(\varphi_H, P)$ draws up to $P$ fresh traces i.i.d. from a declared distribution $\mathcal{D}$ and asks their labels by membership queries. It returns a flag $b$ and a trace $w$: it fails ($b = 0$) as soon as a test disagrees with $\varphi_H$ and returns that trace as the counterexample $w$; if all $P$ tests agree, it succeeds ($b = 1$) without a counterexample, and the guarantee of Theorem 3 applies.

## V. FORMAL PROPERTIES

The identification results concern Algorithm 1 with the exhaustive rule ($K = \infty$), the class schedule by size, no hint pruning, and a comparison budget that is not exhausted; $\mathcal{H}$ denotes the largest searched class $\mathcal{L}_M$, and $\mathcal{H}/\equiv$ its set of equivalence classes. Proofs are in Appendix B.

**Proposition 1** (Sample consistency)**.** *Any hypothesis returned from a nonempty version space agrees with all labels in $S$.*

**Theorem 1** (Conditional bounded identification)**.** *Let $\varphi_T \in \mathcal{H}$, and suppose* DISTINGUISH *returns a witness for every pair of non-equivalent formulas of $\mathcal{H}$. Then Algorithm 1 asks at most $|\mathcal{H}/\equiv| - 1$ refinement queries (line 15) and ends with $\varphi_H \equiv \varphi_T$.*

Under either stopping rule, every query removes at least one equivalence class (Section IV), so a run asks at most $L + |\mathcal{H}/\equiv| - 1 + P$ membership queries, including the initial sample and certification; the comparison budget bounds comparisons, not queries. The implementation bounds DISTINGUISH by a number of product states and, in environments, by a horizon (Section VI); in our experiments, these bounds did not affect any final hypothesis (Section VII-B).

*a) Identification relative to an environment.:* In an environment $\mathcal{E}$, queries are restricted to its feasible traces $L(\mathcal{E}) \subseteq \Sigma^+$, so formulas that differ only on infeasible traces cannot be told apart.

**Definition 3** (Environment-relative equivalence)**.** *Two formulas $\varphi$ and $\psi$ are equivalent relative to an environment $\mathcal{E}$, written $\varphi \equiv_{\mathcal{E}} \psi$, if $L(\mathcal{E}) \cap \big(L(\varphi) \triangle L(\psi)\big) = \emptyset$.*

Equivalence implies $\equiv_{\mathcal{E}}$, but not conversely, so $N_{\mathcal{E}} = |\mathcal{H}/\equiv_{\mathcal{E}}| \leq |\mathcal{H}/\equiv|$. DISTINGUISH then searches for a witness in $L(\mathcal{E}) \cap (L(\varphi) \triangle L(\psi))$.

**Theorem 2** (Conditional environment-relative identification)**.** *Let $\varphi_T \in \mathcal{H}$, and suppose* DISTINGUISH *returns a feasible witness for every pair of formulas of $\mathcal{H}$ that are not equivalent relative to $\mathcal{E}$. Then Algorithm 1 asks at most $N_{\mathcal{E}} - 1$ refinement queries and ends with $\varphi_H \equiv_{\mathcal{E}} \varphi_T$.*

The environment can thus reduce the number of queries. The guarantee needs a complete search over feasible traces: if feasible traces are only sampled, failing to find a witness does not establish $\equiv_{\mathcal{E}}$.

*b) Certification.:* The following bound inverts the binomial probability of observing no errors, as in exact binomial confidence limits [32].

**Theorem 3** (PAC guarantee)**.** *Let $\varphi_H$ be fixed before $P$ tests are drawn i.i.d. from $\mathcal{D}$. With probability at least $1-\delta$ over the tests, some test disagrees with $\varphi_H$, or $\mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) \leq \ln(1/\delta)/P$.*

The same holds for a distribution $\mathcal{D}_{\mathcal{E}}$ supported on $L(\mathcal{E})$, with $\mathrm{err}_{\mathcal{D}_{\mathcal{E}}}(\varphi_H, \varphi_T) = \Pr_{w \sim \mathcal{D}_{\mathcal{E}}}[(w \models \varphi_H) \neq (w \models \varphi_T)]$. Unlike Theorem 2, this bound needs no exhaustive exploration of the environment, but it bounds the disagreement under $\mathcal{D}_{\mathcal{E}}$, not $\equiv_{\mathcal{E}}$.

## VI. IMPLEMENTATION

We implemented Algorithm 1 for LTLf in Python 3.14. Each procedure of Section IV is a separate module, so the enumeration-based and the SAT-based learner share every procedure except VERSIONSPACE. The code and all experiments are available at https://anonymous.4open.science/r/ltl-active-learning-with-hints-585D/. For the pool $T_\ell$ of DISTINGUISH (Section IV-C), we choose $\ell$ as the largest length with at most 5,000 traces in $T_\ell$. If no trace of $T_\ell$ distinguishes the two formulas, the progression-based search runs, bounded by $10^5$ product states and, in environments, by 30 actions; a comparison that reaches this horizon is undecided, and the rival is skipped. For the automata-based search, we build the automata with LTLf2DFA 1.0.2 [33] and MONA 1.4 [34].

All experiments ran on an Apple M4 Pro with 24 GB of memory. The solver is the incremental SAT engine of Z3 5.3.3 [35], used through its Python API; a run is halted after $2 \cdot 10^5$ solver calls or 600 s. The environments use Gymnasium 1.4.0 [36] and MiniGrid 3.2.0 [37].

## VII. EXPERIMENTAL EVALUATION

Our evaluation studies the practical performance of the learner: whether it identifies the target, what identification costs in queries and time, and how natural-language hints change both. All experiments concern LTLf over propositional traces.

TABLE II
BENCHMARK TARGETS. $|S|$: NUMBER OF REACHABLE ENVIRONMENT STATES. SIZES COUNT SYNTAX-TREE NODES, WITH $\neg p$ COUNTED AS ONE NODE.

| **Free traces**, $\mathsf{AP} = \{a, b, c\}$ | |
|---|---|
| Size 2 | $\mathbf{F}a$, $\mathbf{G}a$ |
| Size 3 | $\mathbf{FG}a$, $\mathbf{GF}a$, $a\,\mathbf{U}\,b$ |
| Size 5 | $\mathbf{F}a \to \mathbf{F}b$, $\mathbf{G}(a \to \mathbf{F}b)$, $\mathbf{G}(a \to \mathbf{X}b)$, $\mathbf{F}(a \wedge \mathbf{F}b)$, $a\,\mathbf{U}\,(b\,\mathbf{U}\,c)$, $\mathbf{F}a \wedge \mathbf{G}\neg b$, $\mathbf{F}(a \wedge \mathbf{X}b)$ |
| Size 6 | $\mathbf{G}(a \to b\,\mathbf{U}\,c)$, $a \to \mathbf{X}(b\,\mathbf{U}\,c)$, $\neg a\,\mathbf{U}\,(b \wedge \mathbf{F}c)$, $\mathbf{G}(\neg a \vee \mathbf{XG}\neg b)$ |
| Size 7 | $\mathbf{G}((a \wedge b) \to \mathbf{X}c)$, $\mathbf{GF}a \wedge \mathbf{GF}b$, $\mathbf{G}(a \to \mathbf{X}(a\,\mathbf{U}\,b))$, $\mathbf{F}(a \wedge \mathbf{F}(b \wedge c))$ |
| **Environments** | |
| Office, $\lvert S\rvert = 108$ | $\{f, g, n\}$: $\mathbf{G}\neg n$, $\neg n\,\mathbf{U}\,f$, $\mathbf{F}(f \wedge \mathbf{F}g)$, $\mathbf{F}f \wedge \mathbf{G}\neg n$, $\neg n\,\mathbf{U}\,(f \wedge \mathbf{F}g)$; $\{e, g, n\}$: $\mathbf{F}(e \wedge \mathbf{F}g)$; $\{a, b, c\}$: $\mathbf{F}(a \wedge \mathbf{F}b)$ |
| Taxi, $\lvert S\rvert = 101$ | $\{p, g\}$: $\mathbf{F}g$, $\mathbf{F}(p \wedge \mathbf{F}g)$, $\mathbf{G}(p \to \mathbf{F}g)$ |
| DoorKey 5×5, $\lvert S\rvert = 62$ | $\{k, o, g\}$: $\mathbf{F}g$, $\neg o\,\mathbf{U}\,k$, $\mathbf{F}(o \wedge \mathbf{F}g)$, $\mathbf{G}(k \to \mathbf{F}o)$ |
| DoorKey 6×6, $\lvert S\rvert = 154$ | $\{k, o, g\}$: $\mathbf{F}g$, $\mathbf{F}(k \wedge \mathbf{F}g)$ |

RQ1. **(Identification).** Does the learner return a hypothesis equivalent to the target, and how do the stopping rule and the order of rivals affect this?

RQ2. **(Cost).** How many membership queries and how much time does the learner need, how do they grow with the size of the target, and how much do the order of rivals and the implementation change them?

RQ3. **(Natural-language hints).** How do correct hints affect the number of queries, candidate generation and the running time, and how do incorrect hints affect identification?

### A. Experimental setup

*a) Benchmarks.:* We use two benchmark sets (Table II). The *free* set contains 20 LTLf targets over $\mathsf{AP} = \{a, b, c\}$, so $|\Sigma| = 8$, with sizes 2 to 7 and temporal depth at most 3; here queries may be arbitrary traces. The *environment* set contains 16 targets in four standard benchmarks: the Office world from the reward-machine literature [38], the deterministic version of Taxi [39] from Gymnasium [36], and the DoorKey $5 \times 5$ and $6 \times 6$ layouts of MiniGrid [37]. Each benchmark is represented as a finite deterministic labelled transition system with a fixed start state; states at which the original episode ends are made absorbing. Queries are restricted to feasible traces. Each target uses the propositions listed in Table II; Appendix C describes the environments, their adaptations and the meaning of the propositions.

*b) Learner variants.:* Every run uses the enumeration-based version space with the class schedule by size (Section IV-A) and the progression-based DISTINGUISH after the

short-trace pool $T_\ell$ (Section IV-C). For a target $\varphi_T$ we search $\mathcal{L}_{n,d}$ with $n = |\varphi_T| + 1$ and $d = 2$, or $d = 3$ for the two free targets of temporal depth 3. The bound $n$ is thus assumed to be known up to one, a limitation we return to in Section VIII. INITIALIZE labels $L = 10$ traces: uniformly random traces of length 1 to 6 for free targets, and rollouts with uniformly random actions and 1 to 10 labels for environments. The first hypothesis is a smallest consistent candidate; when a witness refutes the hypothesis, the compared rival replaces it. The comparison budget is 100 (under the exhaustive rule, only comparisons that lead to a query count) and was never exhausted.

A variant of Algorithm 1 is fixed by its stopping rule and its order of rivals (Section IV-B):

- *Stopping rule. Top-*3 is the top-$K$ rule with $K = 3$: a round compares $\varphi_H$ with the three highest-ranked rivals, and refinement ends after a round without a query. *Exhaustive* is $K = \infty$: a round compares $\varphi_H$ with the rivals in order, removes those equivalent to $\varphi_H$ (relative to $\mathcal{E}$ in environments) without a query, and refinement ends only when no remaining candidate is distinguishable from $\varphi_H$. Only the exhaustive rule is covered by the correctness argument of Section IV.
- *Order of rivals. Enumeration* (smallest first); *semantic*, closest first under $d_{\text{sem}}$ on $r = 200$ probe traces (200 rollouts in environments), and its reverse *semantic-far*; and *split*.

RQ1 and RQ2 report these four orders under both rules without hints (1,440 runs); four further orders (1,440 runs) are omitted for space. RQ3 runs enumeration order under both rules with eight hint settings (2,660 runs), one of them repeating the no-hint configuration of RQ1. For RQ2, we also compare two alternative implementations on the free targets under the exhaustive rule: the automata-based DISTINGUISH with MONA, and the SAT-based version space. Each configuration runs with seeds 41 to 45 (100 free and 80 environment runs); the certification results come from a rerun of the 720 top-3 runs that reproduces Table III exactly. Section VI gives the hardware.

*c) Teacher and metrics.:* A simulated teacher answers membership queries by evaluating $\varphi_T$. A run is *successful* (Succ.) if the final hypothesis satisfies $\varphi_H \equiv \varphi_T$ (free targets) or $\varphi_H \equiv_{\mathcal{E}} \varphi_T$ (environments); both are decided exactly by a complete product search that is used only for evaluation. A run is *exact* if $\varphi_H = \varphi_T$ up to notation. MQ counts all labelled traces, including the $L = 10$ initial ones. Eq. counts the comparisons under the exhaustive rule that find a rival equivalent to $\varphi_H$; they remove the rival without a query. Cand. counts the formulas generated by the enumerator, and Time is the wall-clock time of a run.

### B. RQ1: Identification

*a) Exhaustive rule.:* Table III shows that under the exhaustive rule every run succeeds, for every order of rivals: 100 of 100 free and 80 of 80 environment runs. This is the outcome predicted by the correctness argument of Section IV, since every target lies in the searched class $\mathcal{L}_{n,d}$. In environments, the 30-action horizon of DISTINGUISH (Section VI) could leave a distinguishable rival in the version space; it did not affect any final hypothesis. The remaining hypotheses that are not the target formula itself (Exact in Table III) are equivalent to it on the relevant trace domain; in Taxi, for example, $\mathbf{F}g$ and $\mathbf{F}(p \wedge \mathbf{F}g)$ agree on all feasible traces because delivery requires a prior pickup.

*b) Top-*3 *rule.:* Under the top-3 rule, identification depends on which rivals are compared first. On free traces, semantic ordering succeeded in 64 of 100 runs, while semantic-far and split succeeded in all 100. All free-trace failures involved targets of size 5 to 7. In environments, success ranged from 40 to 64 of 80 runs.

Every failed run stopped because none of the three shortlisted rivals yielded a query, even though distinguishable candidates remained. No run exhausted the comparison budget. Closest-first ordering often shortlists equivalent rewritings, which have $d_{\text{sem}} = 0$ on the probe set. Semantic-far and split avoided premature stopping on free traces. Equivalent rivals are more common in environments. In Office world, for example, a formula that requires the agent to start in a labelled cell is equivalent to false on feasible traces. With enumeration order, only 5 of 35 Office world runs succeeded.

*c) Certification.:* Certification detected all identification failures on free traces, but few in environments. We tested each top-3 hypothesis using CERTIFY (Section IV-E). Free traces had geometric lengths with mean 5 and uniform letters. Environment rollouts had geometric step counts with mean 10 and uniform actions. Table III excludes certification queries.

On free traces, certification detected 38 of 44 failures within 30 tests and all 44 within 60 tests ($\epsilon = \delta = 0.05$). In environments, it detected 3 of 135 failures within 60 tests and 21 within 300 ($15/15$ in DoorKey, $6/105$ in Office world, and $0/15$ in Taxi). All equivalent hypotheses passed. For undetected hypotheses, the estimated error under $\mathcal{D}_{\mathcal{E}}$ was at most 0.2% over 20,000 rollouts. Thus, low error under the test distribution does not imply equivalence on feasible traces.

A failed test can be added to $S$ to resume refinement, approximating an equivalence query by random sampling [13]. This strategy offered limited benefit here. On free traces, exhaustive refinement identified every target with at most 2.4 more queries on average than the top-3 rule. In environments, random tests rarely found a counterexample.

*Answer to RQ1.* The exhaustive rule identified every benchmark target up to $\equiv$ on free traces or $\equiv_{\mathcal{E}}$ in environments, regardless of rival order. With the top-3 rule, success ranged from 64 to 100 of 100 free runs and from 40 to 64 of 80 environment runs. Closest-first performed worst on free traces because it often shortlisted rivals equivalent to the hypothesis. In environments, split clearly outperformed the other orders. Certification detected all failures on free traces but few in environments, where random rollouts rarely exposed disagreements.

### C. RQ2: Cost

*a) Queries.:* With the exhaustive rule and enumeration order, a run asks 16.2 queries on average on free targets (stan-

TABLE III

RQ1 AND RQ2: IDENTIFICATION AND COST BY ORDER OF RIVALS AND STOPPING RULE. ALL RUNS USE THE ENUMERATION-BASED VERSION SPACE, THE PROGRESSION-BASED DISTINGUISH AND NO HINTS. TOP-3: TOP-$K$ RULE WITH $K = 3$; EXHAUSTIVE: $K = \infty$. SUCC.: RUNS WITH $\varphi_H \equiv \varphi_T$ (FREE, OUT OF 100) OR $\varphi_H \equiv_{\mathcal{E}} \varphi_T$ (ENVIRONMENTS, OUT OF 80). EXACT: RUNS WITH $\varphi_H = \varphi_T$ UP TO NOTATION; UNDER THE EXHAUSTIVE RULE, 85 FREE AND 30 ENVIRONMENT RUNS FOR EVERY ORDER. MQ: MEAN MEMBERSHIP QUERIES PER RUN, INCLUDING THE 10 INITIAL TRACES; TOP-3 MEANS INCLUDE FAILED RUNS. EQ.: MEAN COMPARISONS PER RUN UNDER THE EXHAUSTIVE RULE THAT FOUND A RIVAL EQUIVALENT TO $\varphi_H$.

| | **Free traces** | | | | | | **Environments** | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Top-3 | | | Exhaustive | | | Top-3 | | | Exhaustive | | |
| **Order of rivals** | Succ. | Exact | MQ | Succ. | MQ | Eq. | Succ. | Exact | MQ | Succ. | MQ | Eq. |
| Enumeration | 92 | 77 | 15.5 | 100 | 16.2 | 14.2 | 40 | 20 | 8.7 | 80 | 12.5 | 310 |
| Semantic (closest) | 64 | 56 | 13.5 | 100 | 15.9 | 20.1 | 40 | 20 | 8.6 | 80 | 12.2 | 310 |
| Semantic (farthest) | 100 | 85 | 16.1 | 100 | 16.1 | 11.9 | 41 | 20 | 9.5 | 80 | 12.5 | 308 |
| Split | 100 | 85 | 15.7 | 100 | 15.7 | 11.8 | 64 | 25 | 10.6 | 80 | 12.4 | 296 |

TABLE IV

RQ2: COST BY TARGET SIZE UNDER THE EXHAUSTIVE RULE WITH ENUMERATION ORDER (NO HINTS). $N$: RUNS; MQ: MEAN (MINIMUM–MAXIMUM); TIME: MEAN SECONDS PER RUN; MAX: MAXIMUM SECONDS PER RUN; CAND.: MEAN NUMBER OF GENERATED FORMULAS, IN THOUSANDS.

| | **Free traces** | | | | | **Environments** | | |
|---|---|---|---|---|---|---|---|---|
| $\lvert\varphi_T\rvert$ | $N$ | MQ | Time | Max | Cand. | $N$ | MQ | Time |
| 2 | 10 | 10.6 (10–13) | 0.01 | 0.03 | 0.2 | 20 | 8.7 (5–11) | 0.02 |
| 3 | 15 | 11.4 (10–13) | 0.01 | 0.02 | 1.0 | 10 | 10.5 (7–13) | 0.02 |
| 5 | 35 | 15.7 (10–20) | 0.10 | 0.11 | 55.8 | 45 | 14.0 (9–21) | 0.19 |
| 6 | 20 | 18.6 (14–22) | 1.07 | 1.32 | 536 | 5 | 18.4 (17–19) | 1.02 |
| 7 | 20 | 20.9 (13–32) | 7.94 | 10.70 | 3,944 | – | – | – |

TABLE V

RQ2: THREE IMPLEMENTATIONS ON THE FREE TARGETS (EXHAUSTIVE RULE, ENUMERATION ORDER, NO HINTS). PROGR.: ENUMERATED VERSION SPACE AND PROGRESSION-BASED DISTINGUISH, AS IN ALL OTHER RUNS; MONA: THE SAME WITH AUTOMATA-BASED DISTINGUISH; SAT: SAT-BASED VERSION SPACE WITH PROGRESSION-BASED DISTINGUISH. TIME: MEAN SECONDS PER RUN; MEM.: MEAN PEAK MEMORY IN MB. EACH RUN HAS ITS OWN PROCESS, WHICH ADDS ABOUT 0.7 S, SO TIMES ARE NOT COMPARABLE WITH TABLE IV. ROWS FOR $n = 9, 10$: THE FOUR SIZE-7 TARGETS WITH SEEDS 41 AND 42 AND A LIMIT OF 600 S (ONE RUN EACH FOR PROGR. AND MONA); THE SAT ENTRY GIVES THE RUNS THAT FINISHED.

| | | **Progr.** | | **MONA** | | **SAT** | |
|---|---|---|---|---|---|---|---|
| $\lvert\varphi_T\rvert$ | $n$ | Time | Mem. | Time | Mem. | Time | Mem. |
| 2 | 3 | 0.83 | 281 | 0.78 | 281 | 0.89 | 315 |
| 3 | 4 | 0.79 | 281 | 0.80 | 281 | 0.98 | 316 |
| 5 | 6 | 0.87 | 290 | 0.97 | 290 | 1.63 | 356 |
| 6 | 7 | 1.81 | 350 | 1.98 | 352 | 3.03 | 415 |
| 7 | 8 | 8.82 | 702 | 9.92 | 728 | 22.35 | 486 |
| 7 | 9 | 70.2 | 3,471 | 69.6 | 3,858 | 2/8 | 579 |
| 7 | 10 | – | – | – | – | 0/8 | 709 |
| Exact | | 85 | | 85 | | 50 | |
| MQ | | 16.2 | | 16.2 | | 16.1 | |

dard deviation 4.5, range 10 to 32) and 12.5 in environments (standard deviation 3.8, range 5 to 21), that is, 6.2 and 2.5 queries beyond the initial sample. Table IV shows that the number of queries grows with the size of the target, from 10.6 for size 2 to 20.9 for size 7 on free targets.

*b) Order of rivals and stopping rule.:* Under the exhaustive rule, the order of rivals changes the mean number of queries by at most 3% on free targets (15.7 for split to 16.2 for enumeration order) and 2% in environments (12.2 to 12.5; Table III). Compared with the top-3 rule under the same order, the exhaustive rule asks 0 to 2.5 more queries on free targets and 1.8 to 3.8 more in environments. These are the queries that the top-3 rule omits; for instance, the low mean of semantic ordering under the top-3 rule (13.5) reflects runs that end early, 36 of them without identifying the target. Equivalence checks cost no queries: the exhaustive rule removes 11.8 to 20.1 equivalent rivals per run on free targets, most under closest-first orders, and 296 to 310 in environments.

*c) Running time.:* The mean running time per run hardly depends on the variant: 1.83 to 1.84 s on free targets and 0.15 to 0.18 s in environments, for every order and both rules. It grows with the size bound $n$, together with the number of generated formulas: for size-7 targets ($n = 8$), the enumerator generates up to 5.1 million formulas and a run takes 7.9 s on average and at most 10.7 s. We did not include targets of size 8. With $n = 9$, a single run (target $\mathbf{G}((a \wedge b) \rightarrow \mathbf{X}c)$, seed 41) generated 31.4 million formulas and took 70 s and 3.4 GB of memory (Table V).

*d) Alternative implementations.:* Table V compares the default implementation with the automata-based DISTINGUISH and the SAT-based version space. All three learn every free target, with 16.1 to 16.2 queries on average. With MONA, the hypotheses and queries are identical in all 100 runs, and runs take at most 13% longer: most comparisons end in the short-trace pool, and only the others build automata. The SAT-based version space is slower at every size, 22.4 s against 8.8 s for size-7 targets. With $n = 9$, only 2 of 8 runs finish within 600 s, and with $n = 10$ none does, although the hypothesis is already equivalent to the target in all 16 runs: the remaining time is spent blocking encodings, since a formula has one encoding per order of its nodes and the exhaustive rule must exclude all of them. In return, the SAT-based construction stores no formulas and needs 0.6 GB with $n = 9$, against 3.4 GB. It also returns the target formula itself less often (50 against 85 of 100 runs), since it does not apply the enumerator's simplifications.

*Answer to RQ2.* The exhaustive rule needs 16.2 queries per

TABLE VI

RQ3: EFFECT OF HINTS. ALL RUNS USE ENUMERATION ORDER OF RIVALS AND THE ENUMERATION-BASED VERSION SPACE. SUCC.: SUCCESSFUL RUNS UNDER THE TOP-3 AND THE EXHAUSTIVE RULE. MQ, CAND. (GENERATED FORMULAS, IN THOUSANDS) AND TIME (SECONDS) ARE MEANS UNDER THE EXHAUSTIVE RULE. ROWS MARKED † COVER ONLY THE TARGETS THAT RECEIVE A LEVEL-2 HINT (70 FREE AND 55 ENVIRONMENT RUNS); NONE† IS THE RUN WITHOUT HINTS ON THESE TARGETS.

| | **Free traces** | | | | | **Environments** | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Succ. | | Exhaustive | | | Succ. | | Exhaustive | | |
| **Hints** | Top-3 | Exh. | MQ | Cand. | Time | Top-3 | Exh. | MQ | Cand. | Time |
| None | 92/100 | 100/100 | 16.2 | 916 | 1.83 | 40/80 | 80/80 | 12.5 | 57 | 0.18 |
| Level 1, ranking | 95/100 | 100/100 | 16.0 | 916 | 1.84 | 64/80 | 80/80 | 12.4 | 57 | 0.20 |
| Level 1, pruning | 94/100 | 100/100 | 14.6 | 710 | 1.56 | 54/80 | 80/80 | 10.5 | 46 | 0.12 |
| None† | 66/70 | 70/70 | 16.0 | 964 | 1.94 | 25/55 | 55/55 | 13.7 | 77 | 0.23 |
| Level 2† | 67/70 | 70/70 | 14.1 | 964 | 1.96 | 25/55 | 55/55 | 11.9 | 77 | 0.23 |
| Level 1 pruning + Level 2† | 67/70 | 70/70 | 14.0 | 964 | 1.96 | 45/55 | 55/55 | 11.9 | 77 | 0.27 |
| Incorrect Level 1, ranking | 93/100 | 100/100 | 15.7 | 916 | 1.84 | 45/80 | 80/80 | 12.6 | 57 | 0.20 |
| Incorrect Level 1, pruning | 67/100 | 74/100 | 13.7 | 369 | 0.76 | 43/80 | 43/80 | 8.8 | 11 | 0.04 |
| Incorrect Level 2 | 53/100 | 58/100 | 15.1 | 1,290 | 2.63 | – | – | – | – | – |

free and 12.5 per environment target on average, growing with target size and barely affected by the order of rivals. Running time grows with the size bound, reaching about a minute and 3.4 GB for $n = 9$; neither alternative implementation lowers this cost.

### D. RQ3: Natural-language hints

*a) Simulated hints.:* At Level 1, the simulated teacher selects the question types $c$ for which $\varphi_T \in \mathcal{H}_c$. It finds a match for 10 of 20 free targets and 14 of 16 environment targets, and answers *unsure* otherwise. At Level 2, it selects the strongest template instance $h$ that is entailed by $\varphi_T$ but is neither valid nor equivalent to $\varphi_T$. Entailment is checked exactly. For example, $\mathbf{G}(a \to \mathbf{X}b)$ receives the hint $\mathbf{G}(a \to \mathbf{F}b)$. Such hints exist for 14 free and 11 environment targets.

For each target, we also generate an incorrect hint at each level. An incorrect Level-1 hint selects a type $c$ with $\varphi_T \notin \mathcal{H}_c$. An incorrect Level-2 hint selects a template instance not entailed by $\varphi_T$, minimising the number of traces in $T_\ell$ that satisfy $\varphi_T$ but violate $h$. We use Level-1 hints for ranking or pruning and Level-2 hints for pruning (Section IV-D), with enumeration as the base rival order and $\ell = 4$ for free targets. We report incorrect Level-2 results only for free traces. In environments, these hints either hold on all feasible traces accepted by the target or have counterexamples only on traces of 14 to 26 labels.

*b) Correct hints.:* Ranking with Level-1 hints barely changes the mean number of membership queries, but raises top-3 success in environments from 40 to 64 of 80 runs by bringing distinguishable rivals into the shortlist (Table VI). Pruning with Level-1 hints reduces membership queries by 10% on free traces (16.2 to 14.6) and 16% in environments (12.5 to 10.5). It also reduces candidate generation by 22% and 20%, respectively, and free-trace running time by 15%.

On targets that receive a Level-2 hint, pruning reduces mean membership queries from 16.0 to 14.1 on free traces and from 13.7 to 11.9 in environments. It does not reduce candidate generation because the hint is checked after enumeration. Combining both levels yields little further query savings, but raises top-3 success in environments from 25 to 45 of 55 runs relative to the no-hint baseline. The exhaustive rule succeeds in every run with correct hints.

*c) Incorrect hints.:* Incorrect ranking hints leave exhaustive identification unchanged (100 of 100 free runs and 80 of 80 environment runs) and change the mean number of membership queries by at most 3%. Incorrect pruning hints can prevent identification even under the exhaustive rule. With Level-1 pruning, success drops to 74 of 100 free runs and 43 of 80 environment runs. With Level-2 pruning, it drops to 58 of 100 free runs.

The search widens only if the current level has no consistent candidate. An incorrect hint can therefore keep the search in a level that contains a sample-consistent formula but no formula equivalent to the target. For example, with $\varphi_T = \mathbf{GF}a$ and the incorrect hint $h = \mathbf{F}a$, the learner returned $\mathbf{FG}a$. The returned formula differs from $\varphi_T$ only on the empty trace.

*Answer to RQ3.* Correct pruning hints reduce queries by up to 16% and candidate generation by about a fifth; ranking improves top-3 identification in environments. Incorrect ranking hints are harmless, but incorrect pruning hints lower exhaustive success to 54–74%.

## VIII. CONCLUSION

We presented an active-learning framework for LTLf in which the teacher only labels traces and answers natural-language questions. The learner identified every benchmark target with few queries, and hints reduced the queries further. Future work includes a study with human users and noisy labels, translating hints with large language models, removing the assumed bound on the target size, and scaling to larger formulas.

## REFERENCES


[1] A. Pnueli, "The temporal logic of programs," in *18th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1977, pp. 46–57.

[2] M. Y. Vardi and P. Wolper, "An automata-theoretic approach to automatic program verification," in *Proceedings of the First Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, 1986, pp. 332–344. [Online]. Available: https://www.cs.rice.edu/~vardi/papers/lics86.pdf

[3] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, pp. 14:1–14:64, 2011.

[4] F. Bacchus and F. Kabanza, "Using temporal logics to express search control knowledge for planning," *Artificial Intelligence*, vol. 116, no. 1–2, pp. 123–191, 2000.

[5] S. A. Seshia, D. Sadigh, and S. S. Sastry, "Toward verified artificial intelligence," *Communications of the ACM*, vol. 65, no. 7, pp. 46–55, 2022.

[6] H. Hasanbeig, D. Kroening, and A. Abate, "Certified reinforcement learning with logic guidance," *Artificial Intelligence*, vol. 322, p. 103949, 2023.

[7] Z. Yang, S. S. Raman, A. Shah, and S. Tellex, "Plug in the safety chip: Enforcing constraints for LLM-driven robot agents," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 435–14 442.

[8] A. Kamath, S. Zhang, C. Xu, S. Ugare, G. Singh, and S. Misailovic, "Enforcing temporal constraints for LLM agents," arXiv:2512.23738, 2025.

[9] H. Kress-Gazit, K. Eder, G. Hoffman, H. Admoni, B. Argall, R. Ehlers, C. Heckman, N. Jansen, R. A. Knepper, J. Křetínský, S. Levy-Tzedek, J. Li, T. D. Murphey, L. D. Riek, and D. Sadigh, "Formalizing and guaranteeing human-robot interaction," *Communications of the ACM*, vol. 64, no. 9, pp. 78–84, 2021.

[10] P. K. Danso, M. S. Hasan, N. Balasubramanian, and O. Chowdhury, "Syntax is easy, semantics is hard: Evaluating llms for LTL translation," in *Proceedings of the 2026 ACM Secure Development Conference (SecDev)*. ACM, 2026, pp. 152–168.

[11] B. Greenman, S. Saarinen, T. Nelson, and S. Krishnamurthi, "Little tricky logic: Misconceptions in the understanding of LTL," *The Art, Science, and Engineering of Programming*, vol. 7, no. 2, 2023.

[12] D. Angluin, "Learning regular sets from queries and counterexamples," *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987.

[13] ——, "Queries and concept learning," *Machine Learning*, vol. 2, no. 4, pp. 319–342, 1988.

[14] F. Vaandrager, "Model learning," *Communications of the ACM*, vol. 60, no. 2, pp. 86–95, 2017.

[15] A. Camacho and S. A. McIlraith, "Learning interpretable models expressed in linear temporal logic," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 29, 2019, pp. 621–630.

[16] I. Gavran, E. Darulova, and R. Majumdar, "Interactive synthesis of temporal specifications from examples and natural language," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 201:1–201:26, 2020.

[17] D. Neider and I. Gavran, "Learning linear temporal properties," in *Formal Methods in Computer Aided Design (FMCAD)*. IEEE, 2018, pp. 1–10. [Online]. Available: https://arxiv.org/abs/1806.03953

[18] R. Roy, Y. Pote, D. Parker, and M. Kwiatkowska, "Learning probabilistic temporal logic specifications for stochastic systems," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, 2025, pp. 4642–4650.

[19] G. De Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2013, pp. 854–860. [Online]. Available: https://www.cs.rice.edu/~vardi/papers/ijcai13.pdf

[20] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, "Patterns in property specifications for finite-state verification," in *Proceedings of the 21st International Conference on Software Engineering (ICSE)*. ACM, 1999, pp. 411–420.

[21] A. Linard and J. Tůmová, "Active learning of signal temporal logic specifications," in *IEEE 16th International Conference on Automation Science and Engineering (CASE)*. IEEE, 2020, pp. 779–785.

[22] R. Raha, R. Roy, N. Fijalkow, and D. Neider, "Scalable anytime algorithms for learning fragments of linear temporal logic," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2022.

[23] W. Luo, P. Liang, J. Du, H. Wan, B. Peng, and D. Zhang, "Bridging LTLf inference to GNN inference for learning LTLf formulae," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 9, 2022, pp. 9849–9857.

[24] R. Roy, J.-R. Gaglione, N. Baharisangari, D. Neider, Z. Xu, and U. Topcu, "Learning interpretable temporal properties from positive examples only," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 5, 2023, pp. 6507–6515.

[25] F. Chiariello, "Learning temporal properties from event logs via sequential analysis," in *31st International Symposium on Temporal Representation and Reasoning (TIME 2024)*, ser. Leibniz International Proceedings in Informatics, vol. 318. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 14:1–14:14.

[26] G. Bombara and C. Belta, "Offline and online learning of signal temporal logic formulae using decision trees," *ACM Transactions on Cyber-Physical Systems*, vol. 5, no. 3, 2021.

[27] D. Li, M. Cai, C.-I. Vasile, and R. Tron, "Learning signal temporal logic through neural network for interpretable classification," in *American Control Conference (ACC)*. IEEE, 2023, pp. 1907–1914.

[28] K. Liang, G. A. Cardona, D. Kamale, and C.-I. Vasile, "Learning optimal signal temporal logic decision trees for classification: A max-flow MILP formulation," in *IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 8332–8337.

[29] N. Baharisangari, Y. Ruan, C. Zhao, and Z. Xu, "A logic-based approach to causal discovery: Signal temporal logic perspective," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, 2025, pp. 4752–4760.

[30] M. O. Rabin and D. S. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.

[31] L. G. Valiant, "A theory of the learnable," *Communications of the ACM*, vol. 27, no. 11, pp. 1134–1142, 1984.

[32] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[33] F. Fuggitti, "LTLf2DFA," Software, Zenodo, 2019. [Online]. Available: https://github.com/whitemech/LTLf2DFA

[34] J. Elgaard, N. Klarlund, and A. Møller, "MONA 1.x: New techniques for WS1S and WS2S," in *Computer Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 1427. Springer, 1998, pp. 516–520.

[35] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems – TACAS 2008*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.

[36] M. Towers, A. Kwiatkowski, J. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, J. Terry, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/d7ff1795e8527f6443371c3933bdb52b-Abstract-Datasets_and_Benchmarks_Track.html

[37] M. Chevalier-Boisvert, B. Dai, M. Towers, R. Perez-Vicente, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/e8916198466e8ef218a2185a491b49fa-Abstract.html

[38] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Reward machines: Exploiting reward function structure in reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022.

[39] T. G. Dietterich, "Hierarchical reinforcement learning with the MAXQ value function decomposition," *Journal of Artificial Intelligence Research*, vol. 13, pp. 227–303, 2000.

## APPENDIX A
## DISTINGUISHING BY FORMULA PROGRESSION

This appendix defines the progression-based DISTINGUISH of Section IV-C2 and proves it correct.

*a) Negation normal form.:* As usual for LTLf [19], we use formulas in negation normal form (NNF), with the weak next $\widetilde{\mathbf{X}}$ ($w, i \models \widetilde{\mathbf{X}}\varphi$ iff $i{+}1 \geq |w|$ or $w, i{+}1 \models \varphi$) and release $\varphi \,\mathbf{R}\, \psi = \neg(\neg\varphi \,\mathbf{U}\, \neg\psi)$ as the duals of $\mathbf{X}$ and $\mathbf{U}$. Under the endpoint semantics of the preliminaries, $\varepsilon \models \neg p$ and $\varepsilon \not\models p$. A formula is *basic* if it is a literal or its root operator is $\mathbf{X}$, $\widetilde{\mathbf{X}}$, $\mathbf{U}$ or $\mathbf{R}$. We write $\mathbf{F}\mathrm{true}$ for $\mathrm{true}\,\mathbf{U}\,\mathrm{true}$, which holds exactly on nonempty words, and $\mathbf{G}\mathrm{false}$ for $\mathrm{false}\,\mathbf{R}\,\mathrm{false}$, which holds exactly on $\varepsilon$.

**Definition 4** (Progression)**.** *For an NNF formula $\varphi$ and a letter $a \in \Sigma$, the formula* $\mathrm{prog}(\varphi, a)$ *is defined by* $\mathrm{prog}(\mathrm{true}, a) = \mathrm{true}$, $\mathrm{prog}(\mathrm{false}, a) = \mathrm{false}$ *and*

$$
\begin{aligned}
\mathrm{prog}(p, a) &= \mathrm{true} \text{ if } p \in a, \text{ and } \mathrm{false} \text{ otherwise},\\
\mathrm{prog}(\neg p, a) &= \mathrm{false} \text{ if } p \in a, \text{ and } \mathrm{true} \text{ otherwise},\\
\mathrm{prog}(\varphi \wedge \psi, a) &= \mathrm{prog}(\varphi, a) \wedge \mathrm{prog}(\psi, a),\\
\mathrm{prog}(\varphi \vee \psi, a) &= \mathrm{prog}(\varphi, a) \vee \mathrm{prog}(\psi, a),\\
\mathrm{prog}(\mathbf{X}\varphi, a) &= \varphi \wedge \mathbf{F}\mathrm{true},\\
\mathrm{prog}(\widetilde{\mathbf{X}}\varphi, a) &= \varphi \vee \mathbf{G}\mathrm{false},\\
\mathrm{prog}(\varphi \,\mathbf{U}\, \psi, a) &= \mathrm{prog}(\psi, a)\\
&\quad \vee \big(\mathrm{prog}(\varphi, a) \wedge \varphi \,\mathbf{U}\, \psi\big),\\
\mathrm{prog}(\varphi \,\mathbf{R}\, \psi, a) &= \mathrm{prog}(\psi, a)\\
&\quad \wedge \big(\mathrm{prog}(\varphi, a) \vee \varphi \,\mathbf{R}\, \psi\big).
\end{aligned}
$$

*For words,* $\mathrm{prog}(\varphi, \varepsilon) = \varphi$ *and* $\mathrm{prog}(\varphi, ua) = \mathrm{prog}(\mathrm{prog}(\varphi, u), a)$. *The* end value $\mathrm{fin}(\varphi) \in \{0, 1\}$ *is* $1$ *for* $\mathrm{true}$, $\neg p$, $\widetilde{\mathbf{X}}\varphi$ *and* $\varphi \,\mathbf{R}\, \psi$; *it is* $0$ *for* $\mathrm{false}$, $p$, $\mathbf{X}\varphi$ *and* $\varphi \,\mathbf{U}\, \psi$; *and it is the minimum (maximum) of the operands' values for* $\wedge$ *(*$\vee$*).*

**Lemma 2** (Progression)**.** *For every NNF formula $\varphi$, letter $a \in \Sigma$ and word $w \in \Sigma^*$: (i) $\varepsilon \models \varphi$ iff $\mathrm{fin}(\varphi) = 1$; and (ii) $aw \models \varphi$ iff $w \models \mathrm{prog}(\varphi, a)$. Consequently, $uw \models \varphi$ iff $w \models \mathrm{prog}(\varphi, u)$ for all $u, w \in \Sigma^*$.*

*Proof.* By induction on $\varphi$, as for progression over infinite traces [4]. Part (i) evaluates the semantic clauses at the end of the word. The only case of (ii) specific to finite words is $\mathbf{X}$ (dually $\widetilde{\mathbf{X}}$): $aw \models \mathbf{X}\varphi$ iff $w \neq \varepsilon$ and $w \models \varphi$, that is, $w \models \varphi \wedge \mathbf{F}\mathrm{true}$. The statement for words follows by induction on $|u|$. □

*b) Normal form.:* To keep progressed formulas from growing, the implementation stores each one in disjunctive normal form over basic formulas: a set of clauses, each a set of basic formulas read as a conjunction ($\mathrm{true}$ is the set containing the empty clause, $\mathrm{false}$ the empty set). The normalisation $\mathcal{N}$ (a) removes clauses that contain $p$ and $\neg p$, (b) removes clauses that contain another clause, and (c) replaces a disjunction with unit clauses $\{p\}$ and $\{\neg p\}$ by $\mathrm{true}$. Rules (a) and (c) are valid over $\Sigma^*$ because exactly one of $p$ and $\neg p$ holds at every position, including the end of the word, so $\mathcal{N}(\chi) \equiv \chi$. We write $\mathrm{prog}_{\mathcal{N}}(\chi, a) = \mathcal{N}(\mathrm{prog}(\chi, a))$ and extend it to words as above.

**Algorithm 2** Progression-based distinguishing procedure DISTINGUISHPROG

**Require:** LTLf formulas $\varphi$ and $\psi$; a fixed order on $\Sigma$
**Ensure:** A shortest word $w \in \Sigma^*$ with $(w \models \varphi) \oplus (w \models \psi)$, or false if none exists

$s \leftarrow (\mathcal{N}(\varphi), \mathcal{N}(\psi))$
**if** $s_1 = s_2$ **then**
  **return** false
**end if**
$Q \leftarrow \langle (s, \varepsilon) \rangle$; $V \leftarrow \{s\}$ ▷ FIFO queue, visited pairs
**while** $Q \neq \langle\rangle$ **do**
  $((s_1, s_2), u) \leftarrow \textsc{Dequeue}(Q)$
  **if** $\mathrm{fin}(s_1) \neq \mathrm{fin}(s_2)$ **then**
    **return** $u$
  **end if**
  **for all** $a \in \Sigma$ in the fixed order **do**
    $t \leftarrow (\mathrm{prog}_{\mathcal{N}}(s_1, a), \mathrm{prog}_{\mathcal{N}}(s_2, a))$
    **if** $t_1 \neq t_2$ and $t \notin V$ **then**
      $V \leftarrow V \cup \{t\}$; $\textsc{Enqueue}(Q, (t, ua))$
    **end if**
  **end for**
**end while**
**return** false

**Lemma 3** (Finitely many progressed formulas)**.** *Let $B(\varphi)$ be the set of basic subformulas of the NNF of $\varphi$, together with* $\mathbf{F}\mathrm{true}$ *and* $\mathbf{G}\mathrm{false}$. *For every $u \in \Sigma^*$, the formula $\mathrm{prog}_{\mathcal{N}}(\mathcal{N}(\varphi), u)$ is a set of subsets of $B(\varphi)$ none of which contains another. Hence at most $2^{2^{|B(\varphi)|}}$ distinct formulas are reached from $\varphi$ by progression, and $|B(\varphi)| \leq 2|\varphi| + 2$.*

*Proof.* By Definition 4, progression only reintroduces the formula itself ($\mathbf{U}$, $\mathbf{R}$), its operand ($\mathbf{X}$, $\widetilde{\mathbf{X}}$), $\mathbf{F}\mathrm{true}$ or $\mathbf{G}\mathrm{false}$, so by induction on $\varphi$ and $|u|$ every progressed formula is a combination of elements of $B(\varphi) \cup \{\mathrm{true}, \mathrm{false}\}$ by $\wedge$ and $\vee$. $\mathcal{N}$ turns it into a set of clauses over $B(\varphi)$ in which, by rule (b), no clause contains another. Each subformula of $\varphi$ yields at most two subformulas of the NNF, one per polarity. □

The bound counts sets of clauses, not languages: equivalent formulas with different normal forms only enlarge the search.

**Lemma 4** (Correctness of DISTINGUISHPROG)**.** *For all LTLf formulas $\varphi$ and $\psi$, Algorithm 2 terminates after at most $2^{2^{|B(\varphi)|}} \cdot 2^{2^{|B(\psi)|}}$ iterations of its main loop. It returns a word if and only if some $w \in \Sigma^*$ satisfies $(w \models \varphi) \oplus (w \models \psi)$, and every returned word is the first such word in length-lexicographic order. In particular, it returns* false *if and only if $\varphi$ and $\psi$ are equivalent over $\Sigma^*$.*

*Proof.* By Lemma 2 and $\mathcal{N}(\chi) \equiv \chi$, a pair $(s_1, s_2)$ enqueued with word $u$ satisfies, for every $v$, $uv \models \varphi$ iff $v \models s_1$ and $uv \models \psi$ iff $v \models s_2$. With $v = \varepsilon$ and part (i), every returned word

distinguishes the formulas; and a pair with equal components has no distinguishing extension, so neither pruning it nor the check in line 2 loses a witness. Each pair is enqueued at most once, so termination follows from Lemma 3. Since the search is breadth-first and extends words in the fixed letter order, the first distinguishing word it reaches is the first in length-lexicographic order. □

*c) Trace domain and environments.:* The search may return $\varepsilon$, a valid query under $\mathcal{W} = \Sigma^*$. In an environment given as a finite labelled transition system, it runs instead over triples of an environment state and two progressed formulas, expands them along the transitions of the environment, and returns nonempty feasible traces. By Lemma 3 there are finitely many triples, and the argument above shows that the search returns a feasible distinguishing trace if and only if one exists. During learning, the number of actions is bounded (Section VI); a search that reaches the bound returns no answer, which preserves soundness but not completeness.

## APPENDIX B
## PROOFS

This appendix proves the results of the main text whose proofs are not given there; each proof refers to the statement it proves.

*Proof of Lemma 1. Automata.* Both translations and the emptiness test terminate. By correctness of the LTLf-to-automaton translation [19], with the stated empty-word convention, and the Boolean product construction [30], $L(\mathcal{A}_\Delta) = \{w \in \Sigma^* \mid (w \models_{\mathrm{LTLf}} \varphi) \oplus (w \models_{\mathrm{LTLf}} \psi)\}$. Therefore, an accepting word is exactly a distinguishing word, and the emptiness test returns false exactly when the formulas are equivalent over finite words.

*Progression.* Algorithm 2 in Appendix A is the search of Section IV-C2. By Lemma 4, it terminates, returns a word if and only if a distinguishing word exists, and returns the first distinguishing word in length-lexicographic order, which is a shortest one. □

*Proof of Proposition 1.* Candidate generation filters by consistency, and selection is restricted to the filtered set. The invariant is restored after every new answer. □

*Proof of Theorem 1.* Every label is given by $\varphi_T$, so the equivalence class of $\varphi_T$ is never removed from the version space. Each query separates $\varphi_H$ from a rival, so the formulas equivalent to one of them become inconsistent: every query removes at least one class, and removing an equivalent rival needs no query. Hence at most $|\mathcal{H}/{\equiv}| - 1$ refinement queries are asked. Under the exhaustive rule, refinement on $\mathcal{H}$ ends only when every remaining formula is equivalent to $\varphi_H$; since the class of $\varphi_T$ remains, $\varphi_H \equiv \varphi_T$. □

*Proof of Theorem 2.* As in the proof of Theorem 1, with $\equiv_\mathcal{E}$ in place of $\equiv$: formulas in one class of $\equiv_\mathcal{E}$ receive the same label on every feasible query, the class of $\varphi_T$ is never removed, and each feasible witness removes at least one class, so at most $N_\mathcal{E} - 1$ refinement queries are asked. At termination no remaining formula can be distinguished from $\varphi_H$ by a feasible trace, so $\varphi_H \equiv_\mathcal{E} \varphi_T$. □

*Proof of Theorem 3.* Let $\epsilon = \ln(1/\delta)/P$. If $\mathrm{err}_\mathcal{D}(\varphi_H, \varphi_T) > \epsilon$, all $P$ independent tests agree with $\varphi_H$ with probability at most $(1-\epsilon)^P \le e^{-\epsilon P} = \delta$. Hence, with probability at least $1-\delta$, some test disagrees or the error is at most $\epsilon$. □

## APPENDIX C
## BENCHMARK ENVIRONMENTS

The environment experiments use four configurations from three benchmark families: Office world, Taxi, and MiniGrid DoorKey. Table VII lists their reachable state counts and the number of target formulas. The target formulas are given in Table II.

TABLE VII
ENVIRONMENT CONFIGURATIONS USED IN THE EXPERIMENTS.

| Environment | States | Targets |
|---|---|---|
| Office world | 108 | 7 |
| Taxi | 101 | 3 |
| DoorKey $5 \times 5$ | 62 | 4 |
| DoorKey $6 \times 6$ | 154 | 2 |

*a) Common representation.:* Each environment is represented as a finite deterministic labelled transition system with a fixed start state. A trace records the label of the initial state followed by the label after each action. Labels are projected onto the propositions used by the target formula. Random rollouts choose actions uniformly. Successful delivery in Taxi and goal states in DoorKey are made absorbing: subsequent actions leave the state and its label unchanged.

*b) Office world.:* The Office world [38] is a $12 \times 9$ grid divided into $3 \times 3$ rooms connected by doorways. The agent starts at $(2, 1)$ and moves up, right, down, or left; a move blocked by a wall leaves it in place. The propositions mark coffee ($f$), mail ($e$), the office ($g$), plants ($n$), and four designated locations ($a, b, c, d$). The seven targets use the proposition sets $\{f, g, n\}$, $\{e, g, n\}$, or $\{a, b, c\}$ and specify avoidance or ordered visits to labelled locations.

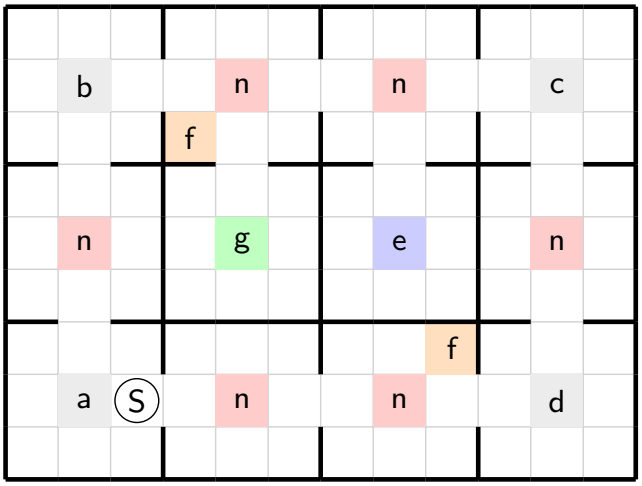



Fig. 1. Office world, drawn from the implemented map. Thick lines denote walls and gaps denote doorways. $S$ marks the start; the other labels identify coffee ($f$), mail ($e$), the office ($g$), plants ($n$), and locations $a, b, c, d$.

*c) Taxi.:* The Taxi benchmark [39] uses Gymnasium's `Taxi-v4` [36], with rain and passenger relocation disabled to obtain deterministic transitions. Reset seed 0 fixes the taxi's initial position, passenger location, and destination. The six actions are south, north, east, west, pickup, and dropoff. Proposition $p$ holds while the passenger is in the taxi, and $g$ holds once the passenger has been delivered. The three targets specify delivery, pickup followed by delivery, or a response requirement for carrying a passenger.

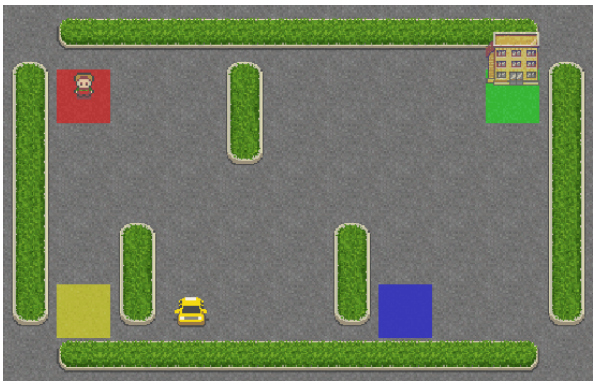
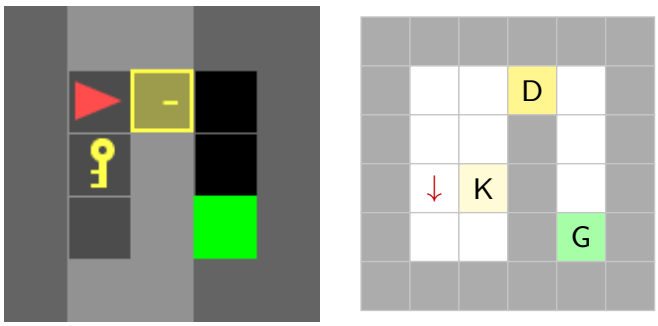


Fig. 2. Illustrative Taxi environment. Image: still frame from the official Gymnasium documentation. The pictured taxi and passenger positions need not match the experiment's reset seed.

*d) MiniGrid DoorKey.:* The DoorKey benchmarks [37] use `MiniGrid-DoorKey-5x5-v0` and `MiniGrid-DoorKey-6x6-v0`, both with reset seed 0. States record the agent's position and orientation, the carried object, and the grid configuration. The available actions are turn left, turn right, move forward, pickup, and toggle; drop and done are omitted, and the original episode step limit is ignored. Proposition $k$ denotes carrying the key, $o$ an open door, and $g$ reaching the goal. The $5 \times 5$ layout has four targets and the $6 \times 6$ layout has two, covering goal reachability and requirements involving the key and door.




(a) DoorKey $5 \times 5$ (b) DoorKey $6 \times 6$

Fig. 3. DoorKey environments. (a) Illustrative $5 \times 5$ layout from the official MiniGrid documentation; its initial configuration may differ from the experiment's. (b) Grid drawn from the $6 \times 6$ experiment with reset seed 0: $K$ denotes the key, $D$ the locked door, $G$ the goal, and the arrow the agent's initial position and orientation. In (a), the key and door are yellow, the goal is green, and the agent is a red triangle.